TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 5: Theoretical Questions

Foreign Trade University

Instructions

Part A contains 17 questions requiring written explanations or short proofs. Part B is a 10-question quiz (true/false and multiple choice). Attempt every question on your own before consulting Part C (Solutions) at the end of this document.

Part A: Theory Questions

A1. State the single-source shortest path problem precisely: what is the input (graph, weights, source), and what is the desired output? Why do we usually require the edge weights to be non-negative for Dijkstra's algorithm?

A2. Describe Dijkstra's algorithm in your own words. What invariant does it maintain about the set S of "finalized" vertices, and what does the relaxation step do to a tentative distance $d[v]$?

A3. Prove that Dijkstra's algorithm is correct on graphs with non-negative edge weights: when a vertex u is added to S, its tentative distance $d[u]$ equals the true shortest-path distance. Identify the exact step in your proof that uses non-negativity.

A4. Analyze the running time of Dijkstra's algorithm implemented with a binary heap. Count the ExtractMin and DecreaseKey/push operations and state the overall bound in terms of n and m.

A5. Give a small directed graph with one negative edge (and no negative cycle) on which Dijkstra returns an incorrect distance. State the wrong answer Dijkstra gives and the correct answer, and explain which step of the correctness proof fails.

A6. State the Bellman–Ford algorithm. Why does it perform exactly $n - 1$ rounds of relaxing all edges, and what does the additional (n-th) round detect?

A7. Compare Dijkstra and Bellman–Ford on (a) the class of graphs they handle correctly, and (b) their running times. When would you choose each?

A8. Define a *spanning tree* and a *minimum spanning tree* of a connected weighted graph. How many edges does a spanning tree of an n-vertex graph have, and why?

A9. Define a *cut* $(S, V \setminus S)$ and what it means for an edge to *cross* the cut. State the cut property.

A10. Prove the cut property: assuming distinct edge weights, the minimum-weight edge crossing any cut belongs to every MST. Use an exchange argument.

A11. State and prove the cycle property: the unique maximum-weight edge on any cycle belongs to no MST.

A12. State Kruskal's algorithm. Prove it correct by showing that every edge it adds is a minimum-weight crossing edge of some cut (and hence safe by the cut property).

A13. State Prim's algorithm. Prove it correct using the cut property, and explain in what sense it is the "graph analogue" of Dijkstra's algorithm.

A14. Describe the union-find (disjoint-set) data structure with *path compression* and *union by rank*. What are the two operations, and what is the amortized cost per operation? How is union-find used inside Kruskal's algorithm?

A15. Suppose the edge weights of a connected graph are all *distinct*. Prove that the minimum spanning tree is unique. What can go wrong (regarding uniqueness) when weights are allowed to be equal?

A16. Using the cut and cycle properties, explain how to decide whether a given edge e belongs to (i) *every* MST, (ii) *some* but not every MST, or (iii) *no* MST. Give a characterization for each case.

A17. Describe single-link k-clustering of maximum spacing and explain its connection to Kruskal's algorithm. What quantity does "stopping Kruskal when k components remain" maximize, and what is the resulting spacing equal to?

Part B: Quiz

B1. (True/False) Dijkstra's algorithm always computes correct shortest-path distances, even when some edge weights are negative (as long as there is no negative cycle).

B2. (Multiple choice) Dijkstra's algorithm implemented with a binary heap runs in time:

(a) $O(n^2)$

(b) $O((m+n)\log n)$

(c) $O(nm)$

(d) $O(m+n)$

B3. (Multiple choice) The Bellman–Ford algorithm runs in time:

(a) $O((m+n)\log n)$

(b) $O(n \log n)$

(c) $O(nm)$

(d) $O(2^n)$

B4. (True/False) The cut property states that the minimum-weight edge crossing any cut belongs to some minimum spanning tree.

B5. (Multiple choice) Kruskal's algorithm uses which data structure to detect whether adding an edge would create a cycle?

(a) A binary min-heap

(b) A union-find (disjoint-set) structure

(c) A stack

(d) A hash map of distances

B6. (True/False) Prim's algorithm and Kruskal's algorithm always produce a spanning tree of the same total weight on a connected graph.

B7. (Short answer) In a connected graph with all distinct edge weights, is the minimum spanning tree unique? (Yes/No, with one sentence why.)

B8. (Multiple choice) By the cycle property, the unique heaviest edge on a cycle belongs to:

(a) Every MST

(b) Some but not every MST

(c) No MST

(d) Exactly one MST

B9. (Short answer) In single-link k-clustering via Kruskal, you stop when how many components remain, and what does the spacing of the clustering equal?

B10. (True/False) Adding the same positive constant to the weight of *every* edge of a graph can change which spanning tree is minimum.

Part C: Solutions

Part A

A1. **Input:** a directed or undirected graph $G = (V, E)$, a weight $w(e) \geq 0$ on each edge, and a source vertex s. **Output:** for every vertex v, the minimum total weight $\text{dist}(v)$ of any s-v path (or ∞ if none exists). We require non-negative weights for Dijkstra because its greedy choice – finalize the nearest unfinished vertex and never revisit it – is only valid when extending a path can never *decrease* its length; a negative edge would let a longer-looking route become cheaper after the vertex is already finalized.

A2. Dijkstra maintains a set S of vertices whose shortest distance is already known, and a tentative distance $d[v]$ for each vertex (the length of the best s-v path found so far). It repeatedly extracts the vertex $u \notin S$ of minimum $d[u]$, finalizes it (adds it to S), and *relaxes* each edge (u, v): if $d[u] + w(u, v) < d[v]$, it lowers $d[v]$ to $d[u] + w(u, v)$ (recording u as v's predecessor). **Invariant:** for every $u \in S$, $d[u]$ equals the true shortest distance to u.

A3. By induction on $|S|$. Base case: $d[s] = 0 = \text{dist}(s)$. Inductive step: let $u \notin S$ be the next extracted vertex, so $d[u] \leq d[v]$ for all $v \notin S$. Every relaxation sets $d[\cdot]$ to a real path length, so $d[u] \geq \text{dist}(u)$. For the reverse, take any s-u path P; it leaves S at a first edge (x, y) with $x \in S$, $y \notin S$. By hypothesis $d[x] = \text{dist}(x)$, and relaxing (x, y) gave $d[y] \leq d[x] + w(x, y)$. The length of P is at least its prefix up to y, which is at least $\text{dist}(x) + w(x, y) \geq d[y] \geq d[u]$ (the last step because u was the minimum). So $\text{dist}(u) \geq d[u]$, hence equality. **Non-negativity is used** at "the length of P is at least its prefix up to y": discarding the edges of P after y can only fail to overcount when those edges have non-negative weight.

A4. There are n ExtractMin operations (one per vertex) and at most m DecreaseKey/push operations (one per edge relaxation). With a binary heap each operation costs $O(\log n)$, so the total is $O((m + n) \log n)$, i.e. $O(m \log n)$ for a connected graph. (A Fibonacci heap improves this to $O(m + n \log n)$.)

A5. Vertices s, a, b; directed edges $s \to a$ (weight 2), $s \to b$ (weight 3), $b \to a$ (weight -2). True $\text{dist}(a) = 1$ via $s \to b \to a$ $(3 - 2)$. Dijkstra extracts a first with $d[a] = 2$, finalizes it, and never revisits it, so it reports 2. The proof step that fails is the use of non-negativity: the path $s \to b \to a$ has a prefix ($s \to b$, length 3) that is *longer* than the whole path, because the final edge is negative.

A6. Bellman–Ford relaxes all m edges, $n - 1$ times. The number $n - 1$ is the maximum number of edges on a simple shortest path (a path visiting n vertices has $n - 1$ edges); after round i, $d[v]$ is correct for all vertices whose shortest path uses at most i edges, so $n - 1$ rounds suffice when no negative cycle exists. The extra (n-th) pass detects negative cycles: if any edge can *still* be relaxed, the graph contains a reachable negative cycle, because a cycle-free shortest path would already have stabilized.

A7. (a) Dijkstra is correct only for non-negative weights; Bellman–Ford handles arbitrary (possibly negative) weights and detects negative cycles. (b) Dijkstra runs in $O((m + n) \log n)$ with a binary heap; Bellman–Ford runs in $O(nm)$. Choose Dijkstra when all weights are non-negative (it is much faster); choose Bellman–Ford when negative edges may be present or when you must detect a negative cycle.

A8. A **spanning tree** of a connected graph G is a subset of edges that connects all n vertices and contains no cycle. A **minimum spanning tree** is a spanning tree of minimum total edge weight. A spanning tree has exactly $n-1$ edges: a connected acyclic graph (a tree) on n vertices always has $n-1$ edges – fewer would leave it disconnected, and one more would create a cycle.

A9. A **cut** is a partition of V into two non-empty parts $(S, V \setminus S)$. An edge **crosses** the cut if it has exactly one endpoint in S (and the other in $V \setminus S$). **Cut property:** the minimum-weight edge crossing any cut belongs to every MST (assuming distinct weights; with ties, it belongs to some MST).

A10. Let $e = (u, v)$ be the min-weight crossing edge of cut $(S, V \setminus S)$, $u \in S$, $v \notin S$, and suppose some MST T omits e. Then $T \cup \{e\}$ has exactly one cycle C, which contains e. A cycle crosses a cut an even number of times, so C has another crossing edge $e' \neq e$; since e is the *minimum* crossing edge and weights are distinct, $w(e) < w(e')$. Then $T' = (T \setminus \{e'\}) \cup \{e\}$ is a spanning tree with $w(T') = w(T) - w(e') + w(e) < w(T)$, contradicting the minimality of T. Hence every MST contains e.

A11. Cycle property: the unique maximum-weight edge f on a cycle C is in no MST. *Proof:* suppose MST T contains $f = (x, y)$. Removing f splits T into two components; let S be the side containing x, so f crosses cut $(S, V \setminus S)$. Cycle C also crosses this cut, so it has another crossing edge $f' \neq f$ with $w(f') < w(f)$ (since f is the unique heaviest on C). Then $(T \setminus \{f\}) \cup \{f'\}$ is a spanning tree of strictly smaller weight, contradicting minimality.

A12. Kruskal: sort edges by increasing weight; for each edge in turn, add it iff its endpoints lie in different components (tested by union-find), and union those components. *Correctness:* when Kruskal adds edge $e = (u, v)$, let S be the set of vertices in u's current component. All edges processed before e are cheaper and were either added (kept inside a component) or discarded, so no cheaper edge crosses the cut $(S, V \setminus S)$; thus e is the minimum-weight crossing edge of this cut, and by the cut property it belongs to every MST. So every edge Kruskal adds is safe; since it stops only when all vertices are connected and never creates a cycle, it outputs a spanning tree of safe edges – the MST.

A13. Prim: start with $S = \{r\}$ for an arbitrary r; repeatedly add the minimum-weight edge with exactly one endpoint in S, and move that endpoint into S, until $S = V$. *Correctness:* at each step Prim adds the minimum crossing edge of the cut $(S, V \setminus S)$, which is safe by the cut property; after $n-1$ additions, T is a spanning tree of safe edges. It is the "graph analogue" of Dijkstra because both grow a single set S outward using a heap, repeatedly pulling the cheapest frontier item – the difference is the key: Prim keys on the *edge weight* crossing the frontier, Dijkstra on the *path distance* $d[u] = d[\text{parent}] + w$.

A14. Union-find maintains a collection of disjoint sets. FIND(x) returns the representative of x's set; UNION(x, y) merges the two sets. *Path compression* makes every node on a find-path point directly at the root; *union by rank* attaches the shorter tree under the taller. With both, the amortized cost per operation is $O(\alpha(n))$ (inverse Ackermann – effectively constant). In Kruskal's algorithm, each vertex starts in its own set; an edge is added iff FIND of its endpoints differ (different components), after which they are UNIONed – this is exactly the cycle-detection test.

A15. Uniqueness with distinct weights: suppose $T_1 \neq T_2$ are both MSTs. Let e be the minimum-weight edge in their symmetric difference (say $e \in T_1 \setminus T_2$); e is unique because

all weights differ. Adding e to T_2 creates a cycle C; C has an edge $e' \notin T_1$ (else T_1 would contain the cycle), and $e' \in T_2 \setminus T_1$ also lies in the symmetric difference, so $w(e) < w(e')$ by choice of e. Then $(T_2 \setminus \{e'\}) \cup \{e\}$ is a spanning tree lighter than T_2 – contradiction. Hence the MST is unique. With equal weights this argument breaks (the "minimum edge in the symmetric difference" need not be unique), and there can be several distinct MSTs of the same total weight (e.g. a triangle with all equal weights has three MSTs).

A16. Compare the global MST weight W with two constrained quantities. (i) e is in *every* MST iff *excluding* e (computing the MST without it) strictly increases the weight above W – equivalently, e is the unique minimum crossing edge of some cut. (ii) e is in *no* MST iff *including* e (forcing it in) gives weight $> W$ – equivalently, e is the unique maximum-weight edge on some cycle (cycle property). (iii) Otherwise (both the forced-in MST and the forced-out MST achieve weight W) e is in *some* but not every MST – this happens with tied weights. Problem 11 implements exactly this three-way test.

A17. Single-link k-clustering of maximum spacing partitions the points into k clusters so as to maximize the *spacing* – the minimum distance between two points in different clusters. The procedure is Kruskal's algorithm stopped once the union-find has k components (equivalently, build the MST and delete its $k-1$ heaviest edges). Stopping when k components remain maximizes the spacing; the resulting spacing equals the weight of the next edge Kruskal would have added – i.e. the $(k-1)$-th most expensive MST edge, the smallest distance between any two of the k surviving components.

Part B

B1. False. Dijkstra can return incorrect distances in the presence of negative edges, even without a negative cycle (it may finalize a vertex before discovering a cheaper route through a negative edge). Use Bellman–Ford instead.

B2. (b) $O((m+n)\log n)$. Each of the n ExtractMin and up to m DecreaseKey/push operations costs $O(\log n)$ with a binary heap.

B3. (c) $O(nm)$. It performs $n-1$ rounds, each relaxing all m edges.

B4. True. (And with distinct weights it belongs to *every* MST; "some MST" is the safe statement that always holds.)

B5. (b) A union-find (disjoint-set) structure. Adding an edge would create a cycle exactly when its endpoints are already in the same set.

B6. True. Both compute a minimum spanning tree, so the total weight is the same (the minimum possible), even if they choose different edge sets when weights are tied.

B7. Yes. With all distinct edge weights the minimum spanning tree is unique (see A15).

B8. (c) No MST. This is the cycle property.

B9. Stop when k components remain; the spacing equals the weight of the next (the $(k-1)$-th heaviest MST) edge Kruskal would have added – the minimum distance between the k resulting clusters.

B10. False. Adding the same constant c to every edge increases every spanning tree's weight by exactly $(n-1)c$ (since each has $n-1$ edges), so the relative order of spanning trees is unchanged and the MST stays the same.